Lecture 13
graphs and design of algorithms

hash table summary:

	search trees	Open Addressing Hash Table
operations	insert, delete, search, find maximum, recall sorted	insert, delete, search
time	$\approx O(\log(n))$ since balanced	$\approx O(1)$
values	comparable values	hashable and check equality

not ordered

Data structures are useful in different settings

rehash: double the number of buckets

linear probing: attempt the next bucket

} good for caching / prefetch

$m > 2n$ so probing works — following pointers is bad for caching
- will find an empty slot

Last Part: Algorithm Design

We analyzed algorithms. How designing new algorithms

Strategy to use:

MIALBET
- make it a little better each time
- selection sort, insertion sort
- merge and partition step

Divide & Conquer
- mergesort and quicksort
- break into smaller parts, solve small part, reassemble

Do slow infrequently
- dynamic array
- only needs to resize once in a while

New: Turn it into a graph

Def: A graph is a set of nodes and edges (connections between nodes)

Weighted: edges have values
Directed: edges have direction
Acyclic: no cycles

n: # of nodes
m: # of edges

sparse: $m \leq n$
dense: $m \geq n^2$

Scenario	nodes / weights	weighted? directed?	sparse/dense
Map	Cities Roads	Distance Direction of road	Dense cycles
Web	Sites URLs	no weights directed	dense cycly
Social Media	users follows	no weight no direction	dense cycly
Courses	Courses prerequisites	directed: order	no cycles
Code	code next line	no weights directed	sparse

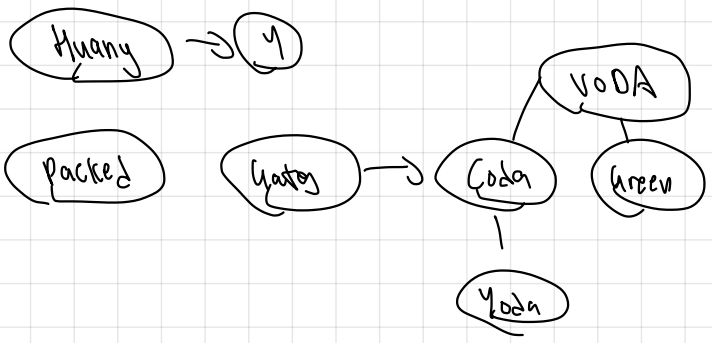
Starting at a node, where can I reach?

Start at Gates. Places I can reach with DFS:

```
def dfs(start_node):
    pre, post ← empty arrays
    visit(start_node, pre, post)
    return pre, post

def visit(node, pre, post):
    if node.visited: return
    node.visited ← true

    pre.push(node)
    for edge in node.outgoing:
        visit(edge.dst, pre, post)
    post.push(node)
```



1. Start at Gates. Visit Gates
2. Visit at Gates neighbors:
3. At Coda, visit all neighbors. Do Voda first
4. At Voda, visit all neighbors. Do green first
5. At Green, no neighbor unvisited. Continue checking Voda's neighbor
6. At Voda, no neighbor unvisited. Continue checking Coda's neighbor
7. At Coda, continue checking neighbor. Do Yoda next.
8. At Yoda, visit all neighbors

Gates
↳ Coda
↳ Voda
↳ Green
↳ Voda ✗
↳ Coda ✗ — visited already
↳ Yoda
↳ Coda ✗ —

Pre: Put on list before visiting all outgoing edges

[Gates, Coda, Voda, Green, Yoda]

Post: Put on list after visiting all outgoing edges

[Green, Voda, Yoda, Coda, Gates]

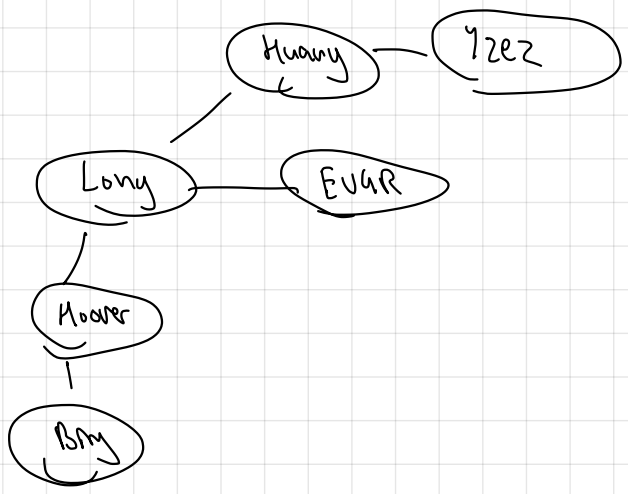
Worst Case Running Time

n nodes
m edges

O(n+m)

each node is called O(1)
each edge is called O(m)

Space is on call stack



Start Yzez:

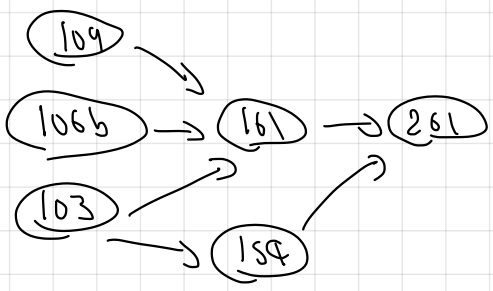
Pre: [Yzez, Huang, Long, Hoover, Bay, EUGR]

Post: [Bay, Hoover, EUGR, Long, Huang, Yzez]

Yzez
↳ Huang
↳ Long
↳ Hoover
↳ Bay
↳ EUGR

toposort algorithm

examples course prerequisites



Given a directed acyclic graph, what is an order to take the classes to not violate prerequisites

APR = [109, 1065, 103]
loop over all classes and find ones with no prerequisites

Loop through APR
- take 1 class out
  - take 103 out and its outgoing edges [103]
  - out of outgoing edges, see if there are new classes without prerequisites, and add to APR
  - 154 no longer has a prerequisite. add to APR
APR = [109, 1065, 154]
- take 1 class out
  - take 109 out [103, 109]
  - no new classes without prerequisites
APR = [1065, 154]
- take 1 class out
  - take 1065 out and its outgoing edges [103, 109, 1065]
  - add 161 to APR
APR = [154, 161]
  - take 154 out [103, 109, 1065, 154]
  - no new additions
APR = [161]
  - take 161 out [103, 109, 1065, 154, 161]
  - add 261
APR = [261]
  - take 261 out [103, 109, 1065, 154, 161, 261]
Done

Worse case running time:

O(n+m)

O(n) first scan
+
O(m) loop through all edges

```
def toposort(graph):
    (make temp copy of graph)

    ordering = []

    all_prereqs_met = set()
    for node in graph:
        if node.incoming_edges is empty:
            all_prereqs_met.insert(node)

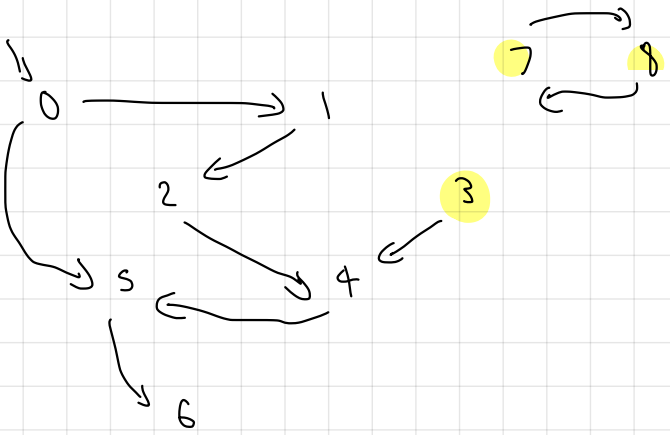
    while all_prereqs_met is not empty:
        take_next = all_prereqs_met.pop()
        ordering.append(take_next)
        for follow_up_class in take_next.outgoing_edges:
            remove edge take_next -> follow_up_class
            if follow_up_class.incoming_edges is empty:
                all_prereqs_met.append(follow_up.class)

    return ordering
```

Code example

```
0 if (x==1) goto line 5
1 x += 1
2 goto line 8
3 y += 1
4 z += 1
5 if (x!=5) goto line 7
6 exit
7 goto line 8
8 goto line 7
```

Each line is a node
Edges are possible control flows



Find dead code:
- Run DFS on the entry of the program (line 0)
- mark nodes visited
- find lines that are not marked visited